

Отчёт: Д34 — технический дизайн микросервисной архитектуры

Дисциплина: Бэк-энд разработка

Выполнил: Губанов Егор, группа БР1.2

Объект: REST API сервиса аренды недвижимости (ЛР1)

1. Объект и контекст

Исходник — монолит в labs/lab1: Express, TypeORM, одна БД PostgreSQL, префикс /api/v1. Контракт API описан в Д32 (TypeSpec → OpenAPI), проверки — Postman из Д33.

Задача Д34: разбить монолит на микросервисы (у каждого своя БД), описать связи между ними, internal-эндпоинты в OpenAPI и шаги переноса.

2. Разбиение на сервисы

Сервис Что внутри

auth регистрация, login, refresh, logout, сброс пароля, /me

catalog типы недвижимости, объекты, фото, условия, поиск

deals сделки, /me/renting, /me/owning/deals

messaging сообщения, прочтение

api-gateway тот же внешний API, что в ЛР1; прокси; сборка /history и /me/owning

Связи между доменами слабые: в чужих БД хранятся только UUID (owner_id, property_id, tenant_id), без FK на чужие таблицы.

3. Отдельные базы данных

БД Таблицы

auth_db users, refresh_tokens, password_reset_tokens

catalog_db property_types, properties, photos, rental_conditions

deals_db deals

messaging_db messages

Пользователей в catalog нет — только owner_id. Сделки и сообщения ссылаются на объект и юзеров по id.

4. Как сервисы общаются

4.1 В ЛР2 — синхронный HTTP

- JSON по REST.

- Внутренние пути: /internal/v1/....

- Заголовок X-Internal-Key (общий секрет в .env).

- Gateway пробрасывает Authorization: Bearer ... на публичные маршруты.

4.2 В Д35 — очереди

Планирую RabbitMQ/Kafka (уведомления, саги). В ЛР2 очередей нет.

4.3 Internal-вызовы

Кто зовёт Кого Зачем

catalog auth GET /internal/v1/users/{id} — owner в карточке

deals catalog GET /internal/v1/properties/{id} — проверка при создании сделки

messaging auth проверка получателя

messaging catalog проверка объекта

gateway deals, messaging, catalog /history, /me/owning

YAML: homeworks/hw4/openapi/.

5. Публичный API

Клиенты ходят на gateway <http://localhost:3000/api/v1> — те же пути и ответы, что в ЛР1 (Postman из ДЗ3 без правок).

6. План переноса с монолита

1. Вынести auth → auth_db.
2. Вынести catalog → catalog_db, owner тянуть из auth по HTTP.
3. Вынести deals → deals_db, property проверять через catalog.
4. Вынести messaging.
5. Поднять gateway, отключить прямой доступ к монолиту.
6. ДЗ5 — очереди; ЛР3 — Docker на все сервисы.

7. Что может пойти не так

Проблема Что делаю

catalog недоступен при создании сделки ошибка 5xx, повтор на клиенте

расхождение OpenAPI и кода сверяю при сдаче ЛР2

медленнее из-за нескольких HTTP в gateway параллельные запросы для history

409 на deals при повторном Postman как в ЛР1 — удаляю старый PENDING

8. Вывод

Спроектированы 4 микросервиса + gateway, у каждого своя БД, internal API в OpenAPI. Реализация — в labs/lab2.